ENTREGA 2B Parte 2: DISEÑO – DIAGRAMAS UML

EQUIPO: Grupo N° 01

2. ETAPA DE DISEÑO

2.1. INTEGRANTES

- 2235610 – Javier Alberto Alean Gonzalez – *Scrum Máster, Development Team*
- 2235607 – William Fernando Basto Corzo– *Development Team*
- 2235656 – David Alejandro Sandoval Rojas – *Product owner, Development Team*

2.2. NOMBRE DEL PROYECTO DEFINIDO

SantanDar



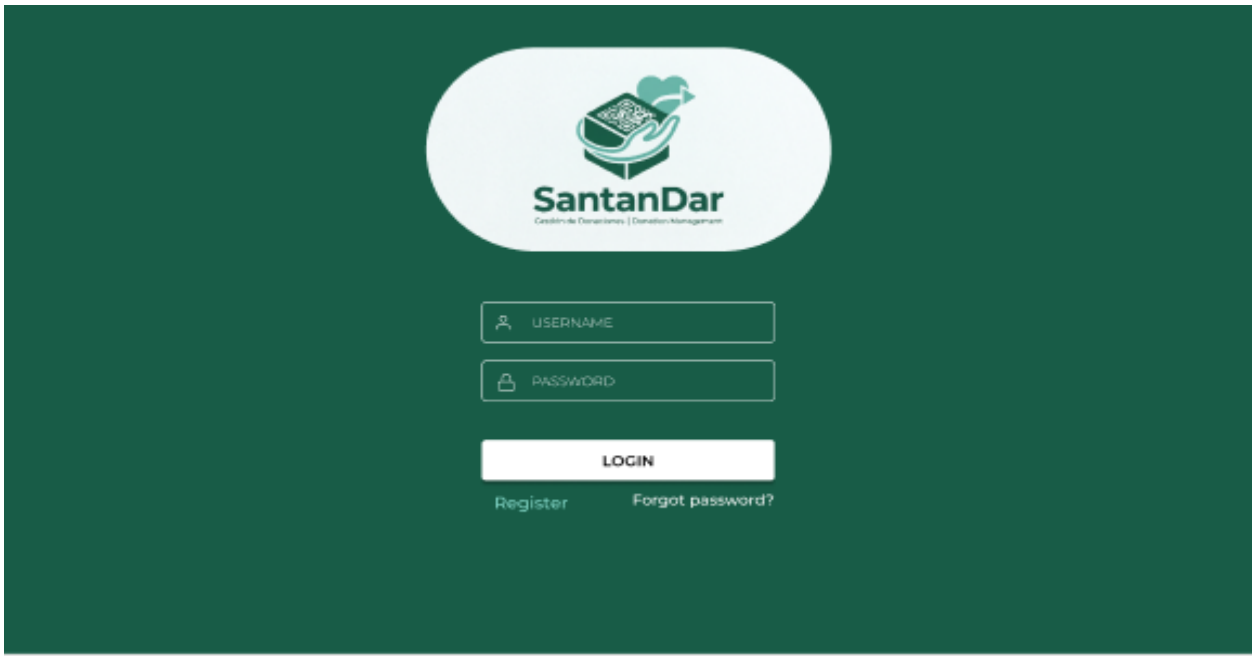
2.3. OBJETIVO GENERAL DEL PROYECTO

Desarrollar una aplicación web para la gestión, control de inventario y trazabilidad logística de los envíos de donaciones en la Universidad Industrial de Santander (UIS), implementando una arquitectura cliente-servidor mediante Java y el framework Spring Boot para el desarrollo del backend, MongoDB Atlas como sistema de gestión de base de datos, y el framework Bootstrap junto con HTML, CSS y JavaScript para el desarrollo de una interfaz responsiva en el frontend, e integrando una API de generación de códigos QR que facilite la identificación y consulta rápida del contenido de las cajas empaquetadas.

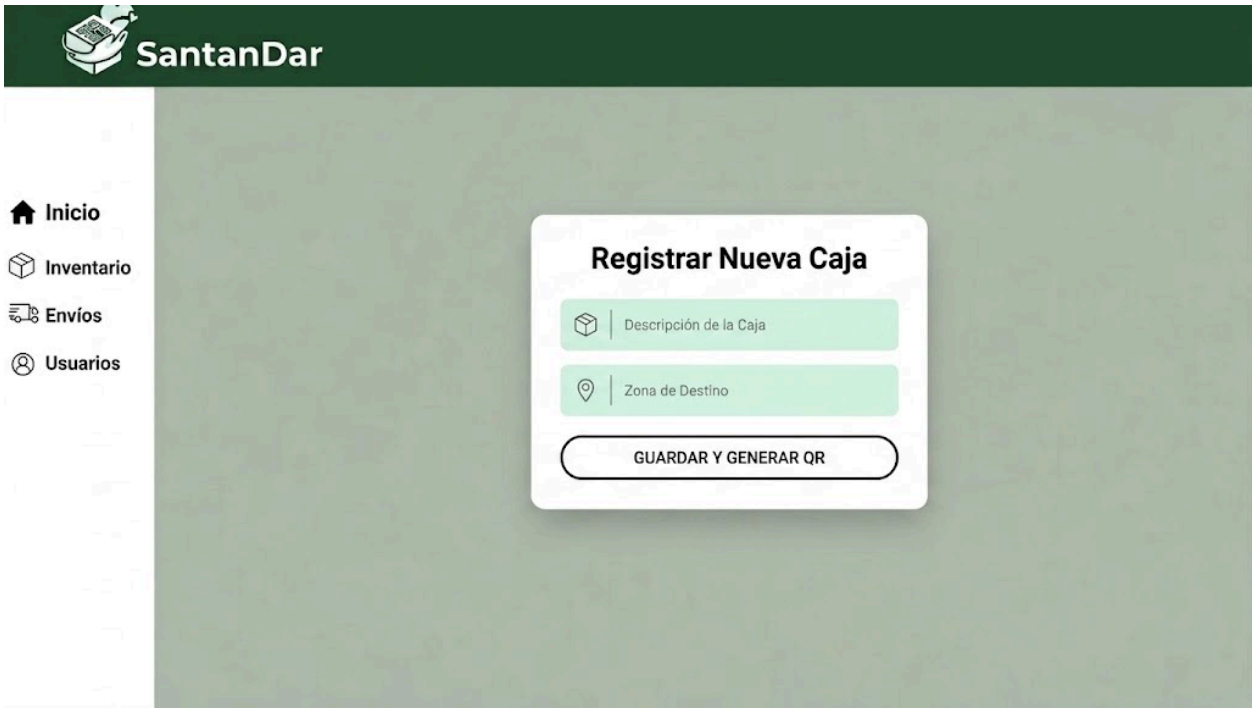
2.4. OBJETIVOS ESPECÍFICOS DEL PROYECTO (MÍNIMO 6):

- **Objetivo 1:** Levantar y documentar los requerimientos funcionales y no funcionales del sistema.
- **Objetivo 2:** Diseñar y modelar la base de datos no relacional en mongoDB ATLAS para estructurar eficientemente el registro de víveres, el armado de cajas y la trazabilidad de los puntos de origen y destino
- **Objetivo 3:** Implementar un módulo de inventario general que permita registrar nuevas cajas con su respectivo contenido y destino
- **Objetivo 4:** Integrar una librería de generación de códigos QR mediante JavaScript que permita asignar y visualizar un identificador único para cada caja empaquetada, facilitando la consulta rápida de su contenido y destino.
- **Objetivo 5:** Desarrollar la lógica del servidor (backend) mediante **Java y el framework Spring Boot**, implementando una API REST que permita gestionar de forma segura y eficiente las operaciones CRUD (Crear, Leer, Actualizar y Borrar) relacionadas con el inventario de recursos.
- **Objetivo 6:** Establecer un sistema de control de acceso (Login) basado en roles, garantizando la integridad de la información y restringiendo la creación de envíos únicamente a los coordinadores autorizados.

2.5. STORYBOARD




SantanDar
INICIO
INVENTARIO
DESPACHO
USUARIOS
Panel de Gestión de Inventario de SantanDar
Añadir Nueva Caja
Total de Cajas Registradas:
312
Buscar...
Filtro por Destino
ID Caja
Destino
Contenido
Estado
Acciones
CAJA-001
Mocoa, Putumayo
Alimentos no perecederos, Agua
En Tránsito
BOX-102
Armero Guayabal, Tolima
Kits de Higiene, Mantas
Preparando
AID-503
Zona Rural, Cauca
Ropa, Medicamentos básicos
Entregado
CAJA-002
Cúcuta, N. de Santander
Alimentos, Kits de Higiene
En Espera
BOX-103
Riosucio, Caldas
Mantas, Ropa, Agua
Preparando
Anterior
1
2
3
...
Next


SantanDar
Inicio
Inventario
Envíos
Usuarios
Registrar Nueva Caja
Descripción de la Caja
Zona de Destino
GUARDAR Y GENERAR QR

Flujo Principal de Gestión de Inventario

1. Interfaz de Autenticación (Login)

- Visual: Pantalla verde institucional con el logo central. Incluye campos de "Usuario", "Contraseña" y botón principal "LOGIN".
- Acción del Usuario: Ingresa credenciales y hace clic en "LOGIN".
- Respuesta del Sistema: Valida la información en la base de datos y transiciona al Dashboard principal.

2. Panel de Gestión (Dashboard de Inventario)

- Visual: Estructura con menú lateral blanco, cabecera verde y área central gris. Muestra indicadores, una tabla con el inventario actual y el botón "Añadir Nueva Caja".
- Acción del Usuario: Hace clic en "Añadir Nueva Caja".
- Respuesta del Sistema: Oculta la tabla de datos y despliega el formulario de registro en la misma vista, manteniendo la navegación intacta.

3. Interfaz "Registrar Nueva Caja"

- Visual: Tarjeta blanca flotante y limpia en el centro. Contiene campos para "Descripción" y "Zona de Destino", junto con el botón "GUARDAR Y GENERAR QR".
- Acción del Usuario: Diligencia la información logística y presiona guardar.
- Respuesta del Sistema: Guarda el registro, ejecuta la generación del código QR y redirige automáticamente al usuario de vuelta a la tabla de inventario actualizada.

2.6. REQUERIMIENTOS NO FUNCIONALES

2.6.1. Listado con la definición de los requerimientos no funcionales según los atributos descritos en la tabla 1).

- **Eficiencia:** RNF1: El sistema debe cargar el panel de control y los resultados de búsqueda en menos de 3 segundos bajo condiciones de conexión a internet estándar (4G o superior).
- **Disponibilidad:** RNF2: El sistema debe estar disponible el 99.9% del tiempo durante las jornadas activas de acopio.
- **Usabilidad:** RNF3: La interfaz de usuario debe ser “responsive” y permitir el escaneo exitoso del código QR.
- **Seguridad:** RNF4: Se debe implementar un sistema de cifrado en tránsito y autenticación mediante tokens para proteger los *endpoints* que modifican el inventario (operaciones exclusivas del Administrador).

- **Concurrencia:** RNF5: El sistema debe ser capaz de manejar simultáneamente al menos 100 usuarios (voluntarios) consultando o escaneando cajas en el mismo instante, sin afectar negativamente el tiempo de respuesta.
- **Mantenibilidad:** RNF6: El código fuente del backend debe estar estructurado en capas (Controladores, Servicios, Repositorios) para facilitar la corrección de errores y futuras actualizaciones.
- **Interoperabilidad:** RNF7: La plataforma debe exponer una arquitectura RESTful que permita en el futuro la integración con sistemas de terceros o nuevas aplicaciones móviles.

2.6.2. Especificación de los requerimientos no funcionales con base en los siguientes atributos de calidad y su respectiva ficha

Especificados en las fichas.

2.6.3. De acuerdo con los requerimientos no funcionales establecidos en el anterior ítem, determine las restricciones técnicas del sistema software web o móvil, en cuánto a las tecnologías que utilizarán como: lenguaje de programación, motor de bases de datos relacionales y no estructuradas, frameworks, middleware, sistema operativo, protocolos de comunicación, librerías, API's interfaz de programación de aplicaciones ya sea externas o desarrolladas

Lenguaje de Programación:

- **Java:** Para el backend. Su tipado fuerte y manejo de hilos nativos aseguran el cumplimiento del requerimiento de Concurrencia (RNF5) y Mantenibilidad (RNF6).
- **JavaScript:** Para el frontend, brindando dinamismo a la interfaz y habilitando el uso del hardware del dispositivo (cámara) para el Escaneo QR.

Motor de Base de Datos:

- **MongoDB Atlas:** Seleccionada por su naturaleza orientada a documentos (JSON). Permite almacenar arreglos dinámicos (como la lista variable de víveres en una caja) ofreciendo alta Eficiencia (RNF1) en lecturas rápidas y Disponibilidad (RNF2) al ser un servicio gestionado en la nube.

Frameworks:

- **Spring Boot (Backend):** Facilita la creación de la API REST (Interoperabilidad - RNF7) con configuraciones embebidas, optimizando los tiempos de despliegue.

- **Bootstrap 5 (Frontend):** Garantiza el cumplimiento directo del requerimiento de *Usabilidad* (RNF3) al proporcionar componentes CSS pre-diseñados y un sistema de grillas 100% responsivo para móviles y tablets.

Middleware y Seguridad:

- **Spring Security con JWT (JSON Web Tokens):** Restricción técnica obligatoria para cumplir con la *Seguridad* (RNF4). Se encargará de interceptar las peticiones y validar los roles (Admin/Voluntario), favoreciendo la escalabilidad.

Sistema Operativo de Despliegue:

- **Windows:** Se seleccionó este entorno operativo porque es la plataforma de mayor dominio técnico por parte del equipo de desarrollo. Trabajar sobre un sistema con el que el equipo ya está altamente familiarizado garantiza una mayor Confiabilidad y Mantenibilidad (RNF6).

Protocolos de Comunicación:

- **HTTPS:** Cifrado obligatorio para proteger la transmisión de credenciales y tokens.
- **HTTP/REST:** Para la comunicación estándar cliente-servidor mediante JSON.

Librerías y APIs Externas:

- **Librería QR Frontend(qrcode.js) :** Restricción técnica para habilitar la lectura de códigos directamente desde el navegador del dispositivo móvil del voluntario sin necesidad de instalar aplicaciones nativas.

Tabla 1. Atributos de calidad para las restricciones del sistema software

Atributos	Descripción
Eficiencia	Conjunto de atributos que relacionan el desempeño y la cantidad de recursos necesitados bajo condiciones establecidas (*Ejemplo: El sistema debe cargar una página web en menos de 2 segundos*).
Fiabilidad	Conjunto de atributos relacionados con la capacidad del software de mantener su nivel de prestación bajo condiciones establecidas durante un periodo de tiempo establecido (*Ejemplo: El sistema software deberá estar disponible el 99.9% del tiempo durante un período de un año, excluyendo el tiempo de mantenimiento programado*).
Confiabilidad	Este término es necesario sea separado en varios elementos que permiten darle al software el matiz de fiable. Sus componentes son: Completitud, Consistencia y precisión, Solidez, Simplicidad, Calidad en los procesos de desarrollo, Seguridad y

	Verificabilidad, estas dos últimas que se determinan con el sistema en uso. (*Ejemplo: El sistema software debe garantizar la integridad de los datos con un factor de redundancia de al menos 3 ubicaciones separadas y recuperables*)
Disponibilidad	Grado en el cual un sistema o componente es operacional y accesible cuando es requerido para su uso. (*Ejemplo: El sistema debe estar disponible las 24 horas del día, los 7 días de la semana, con un tiempo de inactividad máximo de 30 minutos al mes*)
Usabilidad	Conjunto de atributos relacionados con el esfuerzo necesario para el uso, y en la valoración individual de tal uso, por un establecido o implicado conjunto de usuarios. (*Ejemplo: La interfaz de usuario debe ser intuitiva y fácil de usar, con tiempos de respuesta que no superen los 3 segundos*)
Mantenibilidad	Conjunto de atributos relacionados con la facilidad de extender, modificar o corregir errores en un sistema o software (*Ejemplo: El software debe ser fácil de mantener y actualizar, con documentación detallada para los desarrolladores y administradores*).
Flexibilidad	Facilidad con la que un sistema o componente puede ser modificado para ser usado en ambientes diferentes para los cuales fue diseñado inicialmente (*Ejemplo: El sistema software deberá ser compatible con al menos tres sistemas de gestión de bases de datos diferentes (por ejemplo, MySQL, PostgreSQL y Microsoft SQL Server*)
Extensibilidad	Consiste en la facilidad de adaptación del software a nuevos requisitos o cambios en la especificación. REUTILIZACIÓN es otro factor de calidad que consiste en crear elementos de software que sirvan para construir distintas aplicaciones (*Ejemplo: El sistema software debe ser diseñado de manera que permita la incorporación de nuevas funcionalidades y módulos de forma modular y sin afectar las partes existentes del sistema*).
Seguridad	Propiedad de un sistema para salvaguardar la privacidad e integridad de la información. (*Ejemplo: Se debe implementar un sistema de autenticación de dos factores para el acceso de los usuarios*)
Integridad	No ocurrencia de alteraciones no autorizadas de información (*Ejemplo: El sistema de gestión de documentos debe implementar un control de versiones que garantice la integridad de los documentos almacenados*).
Interoperabilidad	Es la condición mediante la cual sistemas heterogéneos pueden intercambiar procesos o datos (*Ejemplo: El sistema debe ser compatible con sistemas y plataformas externas, como servicios web, sistemas de terceros o aplicaciones móviles*)
Concurrencia	Capacidad que tiene un sistema para que ingresen múltiples usuarios al mismo tiempo (*Ejemplo: El sistema software debe ser capaz de manejar simultáneamente al menos 200 transacciones en el mismo instante, sin afectar negativamente el rendimiento y la respuesta del sistema*)
Accesibilidad	Éste no se refiere a la facilidad de uso, sino a la posibilidad de acceso. En concreto a que el diseño, como prerrequisito imprescindible para ser usable, posibilite el acceso a todos sus potenciales usuarios, sin excluir a aquellos con limitaciones individuales (discapacidades, dominio del idioma, etc.) (*Ejemplo: Debe ser accesible para personas con discapacidades de acuerdo con las pautas de accesibilidad WCAG*)

ID	*RNF1*	**Fuente**	*Equipo de Desarrollo / Arquitectura*		
Nombre	*Tiempo de respuesta óptimo en consultas de inventario*				
Complejidad	*Media*		**Prioridad**	*5*	
Tipo	*Necesario*		**Requerimiento que lo utiliza o especializa**	*RF4, RF5, RF6*	
Crítico	*Sí. La lentitud en la carga retrasaría la logística en el centro de acopio.*				
Documentos de visualización asociados	*N/A*				
Usuarios	*Administrador y Voluntario*				
Entrada	*Petición GET al servidor*		**Salida**	*Carga completa del DOM e información JSON*	
Descripción					
El sistema debe resolver las consultas a la base de datos y renderizar el Dashboard o el detalle del escaneo QR en un tiempo inferior a 3 segundos.					
Precondición					
El servidor debe estar en línea y el usuario contar con conexión a red					
Postcondición					
Los datos se visualizan íntegramente en la pantalla del dispositivo					
Consideraciones					
MANEJO DE SITUACIONES ANORMALES: Si la latencia de red del usuario es alta, el sistema debe mostrar un indicador visual de carga (spinner).					
Criterios de aceptación					
Las pruebas de carga mediante herramientas como Postman o JMeter arrojan un promedio de respuesta de la API menor a 2000 milisegundos.					

ID	*RNF2*	**Fuente**	*Equipo de Desarrollo / Product Owner*		
Nombre	*Alta disponibilidad del sistema en jornadas de acopio*				
Complejidad	*Media*		**Prioridad**	*5*	

Tipo	*Necesario*	**Requerimiento que lo utiliza o especializa**	*RF1 a RF10*
Crítico	*Sí. Si el sistema se cae durante una jornada, se detiene el registro y despacho de cajas.*		
Documentos de visualización asociados	*N/A*		
Usuarios	*Administrador y Voluntario*		
Entrada	*Petición de acceso al sistema mediante URL*	**Salida**	*Despliegue de la aplicación web funcional*
Descripción			
El sistema debe estar disponible el 99.9% del tiempo durante las jornadas activas de acopio, excluyendo el tiempo de mantenimiento programado.			
Precondición			
El servidor y los servicios de red locales deben estar encendidos y configurados correctamente.			
Postcondición			
El sistema opera continuamente permitiendo la gestión del inventario.			
Consideraciones			
MANEJO DE SITUACIONES ANORMALES: En caso de caída abrupta del proceso Java, el sistema operativo del servidor debe contar con un script o servicio configurado para reiniciar automáticamente Spring Boot.			
Criterios de aceptación			
Los registros del servidor evidencian un tiempo de inactividad menor a 43 minutos mensuales (cumpliendo el 99.9% de uptime).			

ID	*RNF3*	**Fuente**	*Usuarios finales (Voluntarios)*
Nombre	*Interfaz responsiva y escaneo ágil de QR*		
Complejidad	*Baja*	**Prioridad**	*5*
Tipo	*Necesario*	**Requerimiento que lo utiliza o especializa**	*RF4, RF5, RF6*
Crítico	*Sí. El personal logístico utilizará dispositivos móviles (celulares) para escanear y visualizar el dashboard mientras se mueven por el punto de acopio.*		

Documentos de visualización asociados	*Storyboard y diseño de interfaces visuales*		
Usuarios	*Administrador y Voluntario*		
Entrada	*Interacción táctil en la pantalla*	**Salida**	*Activación de la cámara o retroalimentación visual inmediata*
Descripción			
La interfaz de usuario debe ser responsiva (adaptable a dispositivos móviles) y permitir el escaneo exitoso del código QR en un máximo de 3 pasos intuitivos.			
Precondición			
Acceder a la plataforma desde un navegador móvil con Bootstrap habilitado.			
Postcondición			
El voluntario completa la consulta ágilmente sin requerir capacitación compleja.			
Consideraciones			
MANEJO DE SITUACIONES ANORMALES: Si el dispositivo del usuario no cuenta con cámara funcional o niega los permisos, el sistema mostrará un mensaje amigable sugiriendo usar la búsqueda manual por ID			
Criterios de aceptación			
La interfaz aprueba validaciones de diseño responsivo y un usuario promedio logra abrir el escáner y leer un QR en 3 toques o menos.			

ID	*RNF4*	**Fuente**	*ProductOwner*
Nombre	*Protección de Endpoints mediante Tokens y Cifrado*		
Complejidad	*Alta*	**Prioridad**	*5*
Tipo	*Necesario*	**Requerimiento que lo utiliza o especializa**	*RF1, RF2, RF7, RF8*
Crítico	*Sí. Un acceso no autorizado podría alterar o borrar el inventario de ayudas.*		
Documentos de visualización asociados	*N/A*		
Usuarios	*Administrador*		

Entrada	*Credenciales de acceso / Token en la cabecera HTTP*	**Salida**	*Acceso concedido (200 OK) o denegado (401/403)*
Descripción			
El sistema salvaguardará la integridad de la información exigiendo un token de autorización válido para cualquier operación de escritura en la base de datos			
Precondición			
El usuario debe iniciar sesión exitosamente para obtener su token			
Postcondición			
Se procesa la transacción de inventario de forma segura.			
Consideraciones			
MANEJO DE SITUACIONES ANORMALES: Si el token expira o es manipulado, el sistema rechazará la petición y redirigirá al usuario al Login.			
Criterios de aceptación			
El sistema bloquea con un error HTTP 403 Forbidden cualquier intento de enviar peticiones POST, PUT o DELETE sin un token válido de Administrador.			

ID	*RNF5*	**Fuente**	*Arquitectura de Software*	
Nombre	*Soporte de múltiples usuarios simultáneos*			
Complejidad	*Media*		**Prioridad**	*4*
Tipo	*Necesario*		**Requerimiento que lo utiliza o especializa**	*RF4, RF5, RF6*
Crítico	*Sí. Durante crisis humanitarias, muchos voluntarios y coordinadores usarán la plataforma al mismo tiempo.*			
Documentos de visualización asociados	*N/A*			
Usuarios	*Voluntario y Administrador*			
Entrada	*Múltiples peticiones HTTP concurrentes al servidor*	**Salida**		*Respuestas JSON procesadas correctamente y en orden*
Descripción				

El sistema debe ser capaz de manejar simultáneamente al menos 100 usuarios consultando o escaneando cajas en el mismo instante, sin afectar negativamente el rendimiento de la base de datos.
Precondición
El servidor y la base de datos MongoDB Atlas deben tener ancho de banda y conexiones de red estables.
Postcondición
El sistema resuelve todas las solicitudes sin perder información ni rechazar conexiones válidas.
Consideraciones
MANEJO DE SITUACIONES ANORMALES: Si el pico de usuarios supera el límite del servidor, el sistema debe encolar las peticiones o mostrar un mensaje de "Sistema ocupado, reintente en segundos" para evitar su caída total.
Criterios de aceptación
Las pruebas de estrés confirman que el backend en Spring Boot soporta más de 100 hilos (threads) activos procesando lecturas simultáneamente.

ID	*RNF6*	**Fuente**	*Equipo de Desarrollo*		
Nombre	*Arquitectura de código estructurada en capas*				
Complejidad	*Media*		**Prioridad**	4	
Tipo	*Necesario*		**Requerimiento que lo utiliza o especializa**	*N/A (Aplica al código en general)*	
Crítico	*No bloquea el uso inmediato, pero es estrictamente necesario para el mantenimiento a largo plazo y para cumplir con los estándares académicos de la materia.*				
Documentos de visualización asociados	*N/A*				
Usuarios	*Equipo de Desarrollo*				
Entrada	*Incorporación de nuevo código o módulos*		**Salida**	*Despliegue sin afectar funciones existentes*	
Descripción					
El código fuente del backend (Java) debe estar estructurado en capas (Controladores, Servicios, Modelos, Repositorios) para facilitar la corrección de errores y escalabilidad del software.					
Precondición					
El código fuente del backend (Java) debe estar estructurado en capas (Controladores, Servicios, Modelos, Repositorios) para facilitar la corrección de errores y escalabilidad del software.					

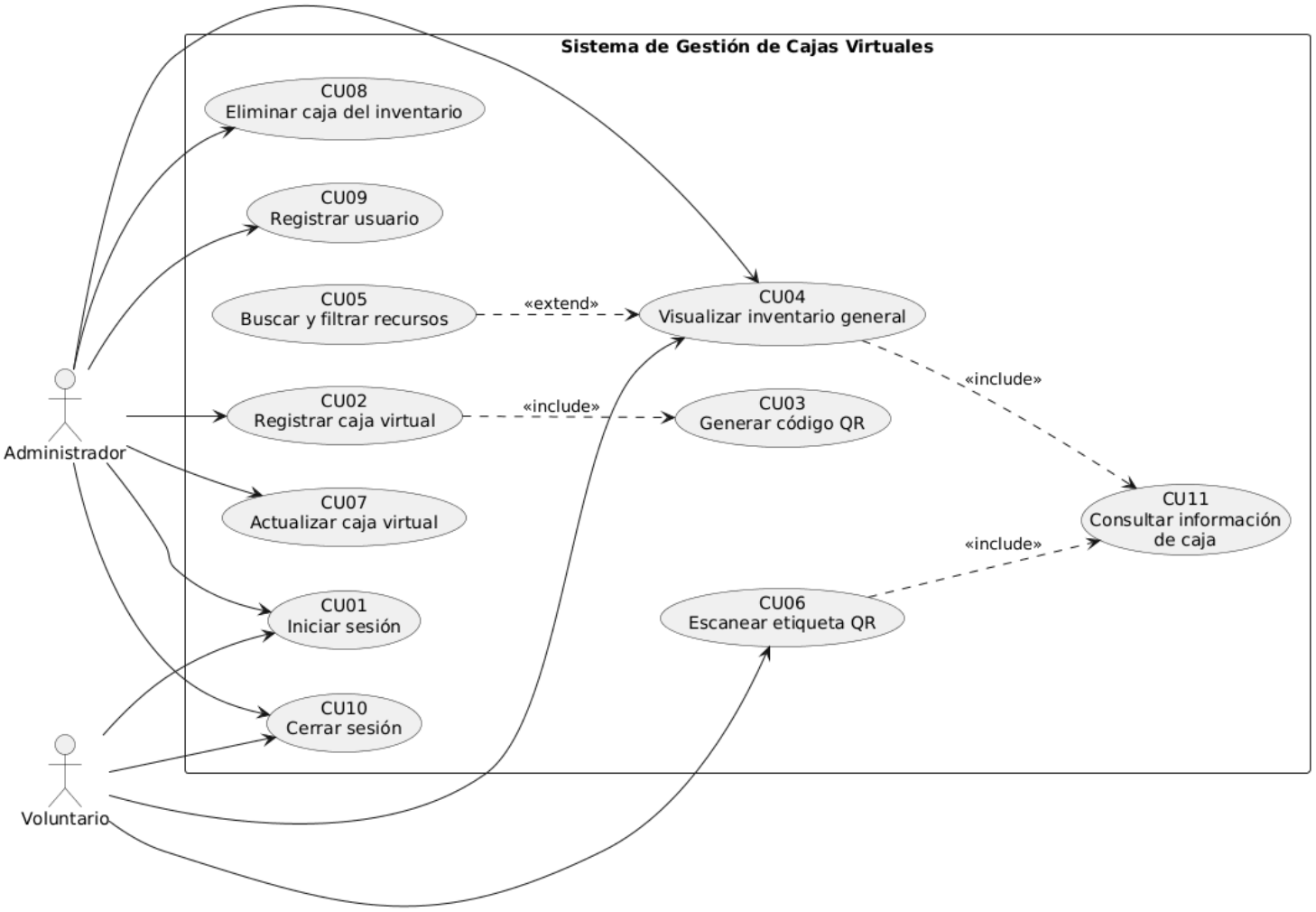
Postcondición
El código base permanece organizado, modularizado y comprensible.
Consideraciones
MANEJO DE SITUACIONES ANORMALES: Si ocurre un error lógico en la capa de datos (Repositorio), la separación de capas evitará que toda la aplicación colapse, aislando la falla.
Criterios de aceptación
La revisión del repositorio de código demuestra una clara separación de paquetes y responsabilidades, sin mezclar lógica de negocio con controladores web.

ID	*RNF7*	**Fuente**	*Arquitectura de Software*		
Nombre	*Exposición de servicios mediante API RESTful*				
Complejidad	*Media*		**Prioridad**	3	
Tipo	*Deseable*		**Requerimiento que lo utiliza o especializa**	*RF1 a RF10*	
Crítico	*No es crítico para la primera entrega, pero es fundamental para la flexibilidad de integrar el sistema con otras plataformas a futuro.*				
Documentos de visualización asociados	*N/A*				
Usuarios	*Sistemas externos / Equipo Frontend*				
Entrada	*Peticiones HTTP estándar (GET, POST, PUT, DELETE)*		**Salida**	*Formato de intercambio de datos estrictamente en JSON*	
Descripción					
La plataforma debe exponer una arquitectura RESTful basada en Endpoints estandarizados que permita intercambiar datos con plataformas externas o aplicaciones de terceros.					
Precondición					
Diseño adecuado de las rutas y controladores (ej. /api/v1/cajas).					
Postcondición					
La aplicación es fácilmente consumible desde cualquier cliente o lenguaje.					
Consideraciones					
MANEJO DE SITUACIONES ANORMALES: Si un cliente externo envía una estructura de datos incorrecta, la API responderá con un código HTTP 400 Bad Request indicando el error de sintaxis.					
Criterios de aceptación					

Todas las interacciones de datos del proyecto SantanDar se pueden probar y operar mediante herramientas de testing de APIs (como Postman o Swagger) usando el formato JSON

2.7. DISEÑO UML

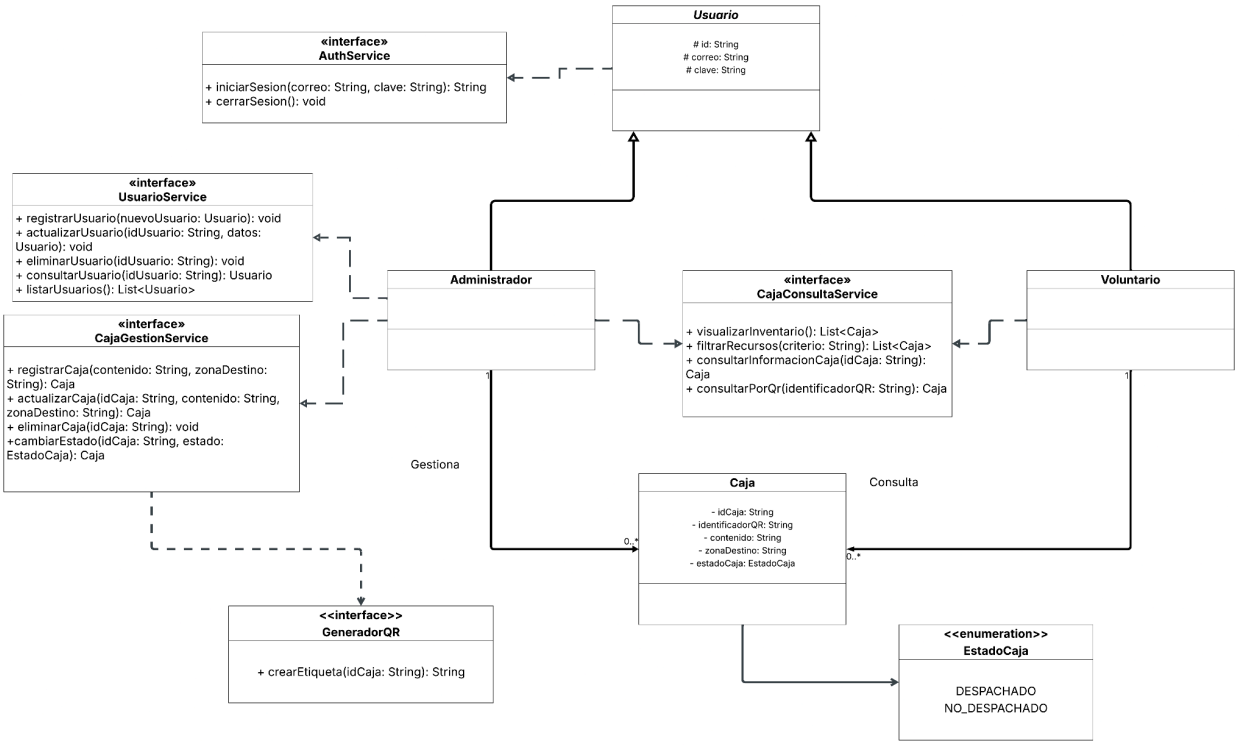
2.7.1. Elaborar el diagrama de casos de uso del sistema propuesto

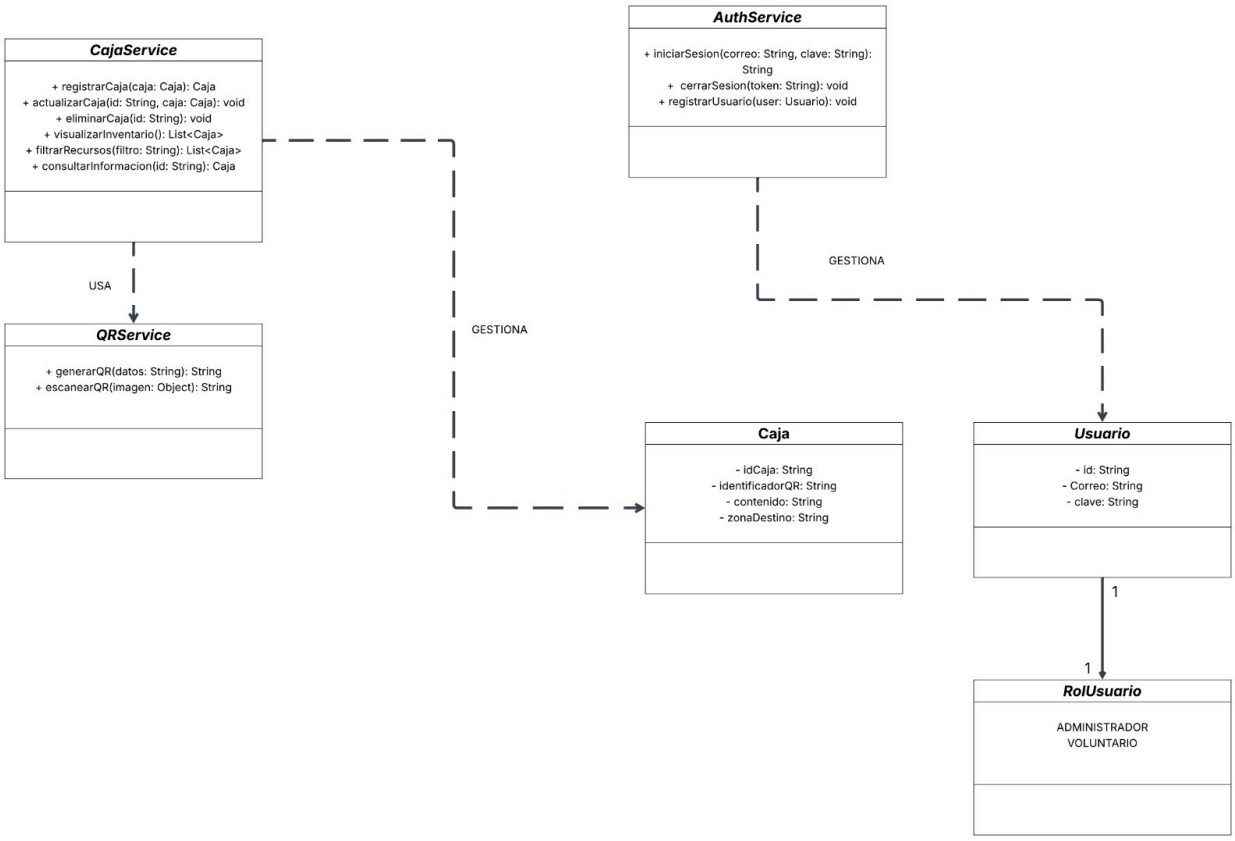


2.7.2. Elaborar el diagrama de clases donde se describen las Clases, Interfaces, Relaciones entre clases, Paquetes, Cardinalidad, propuesto como solución estructural para su proyecto o producto software.

Universidad Industrial de Santander

INGENIERÍA DE SOFTWARE I – 22969 – C2





CRITERIO DE EVALUACIÓN: Enfoque Arquitectónico

- Propuesto 1 (Seleccionado): Basado en interfaces y principios SOLID (Segregación de Interfaces).
- Propuesto 2 (Alternativa): Monolítico / Servicios consolidados directos.

CRITERIO DE EVALUACIÓN: Seguridad de Roles

- Propuesto 1 (Seleccionado): Alta. Interfaces separadas para Administrador y Voluntario.
- Propuesto 2 (Alternativa): Baja. Un solo servicio expone todas las operaciones a cualquier usuario.

CRITERIO DE EVALUACIÓN: Mantenibilidad

- Propuesto 1 (Seleccionado): Alta. Fácil de cambiar, probar o actualizar módulos sin romper otros.
- Propuesto 2 (Alternativa): Media/Baja. Cualquier cambio afecta a un servicio grande y consolidado.

CRITERIO DE EVALUACIÓN: Modelo de Dominio

- Propuesto 1 (Seleccionado): Incluye herencia de usuarios.
- Propuesto 2 (Alternativa): los roles se manejan de forma simple.

CRITERIO DE EVALUACIÓN: Complejidad Inicial

- Propuesto 1 (Seleccionado): Requiere más clases e interfaces en la fase inicial.
- Propuesto 2 (Alternativa): Menos componentes, desarrollo inicial más rápido pero difícil de escalar.

VENTAJAS Y DESVENTAJAS

PROPUESTO 1 (SELECCIONADO):

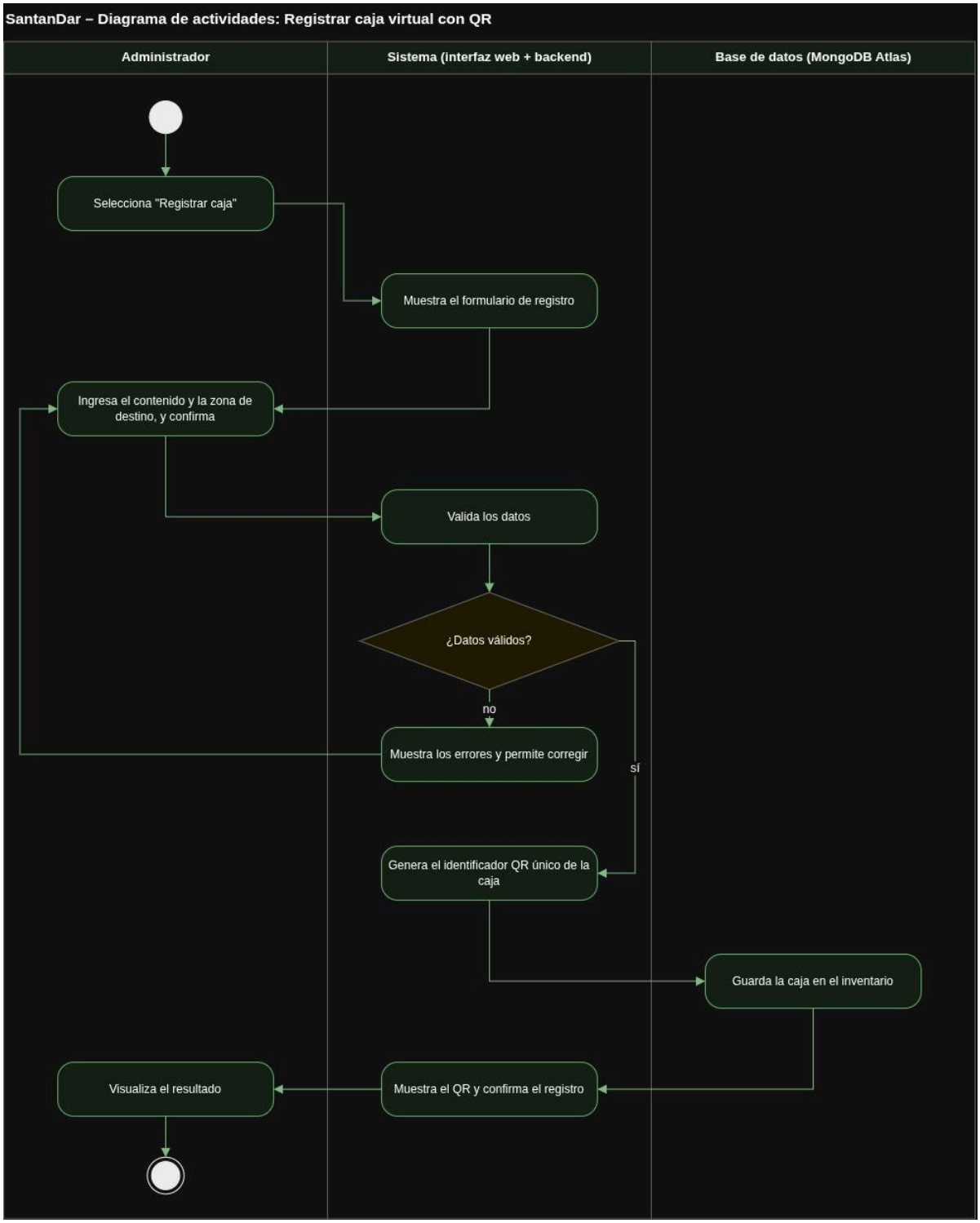
- Ventajas: Gran control de permisos por rol, fácil de mantener a futuro y seguro.
- Desventajas: Requiere un poco más de tiempo de desarrollo al principio.

PROPUESTO 2 (ALTERNATIVA):

- Ventajas: Estructura inicial más simple y rápida de programar.
- Desventajas: Mezcla permisos de usuarios, difícil de escalar y faltan datos del negocio.

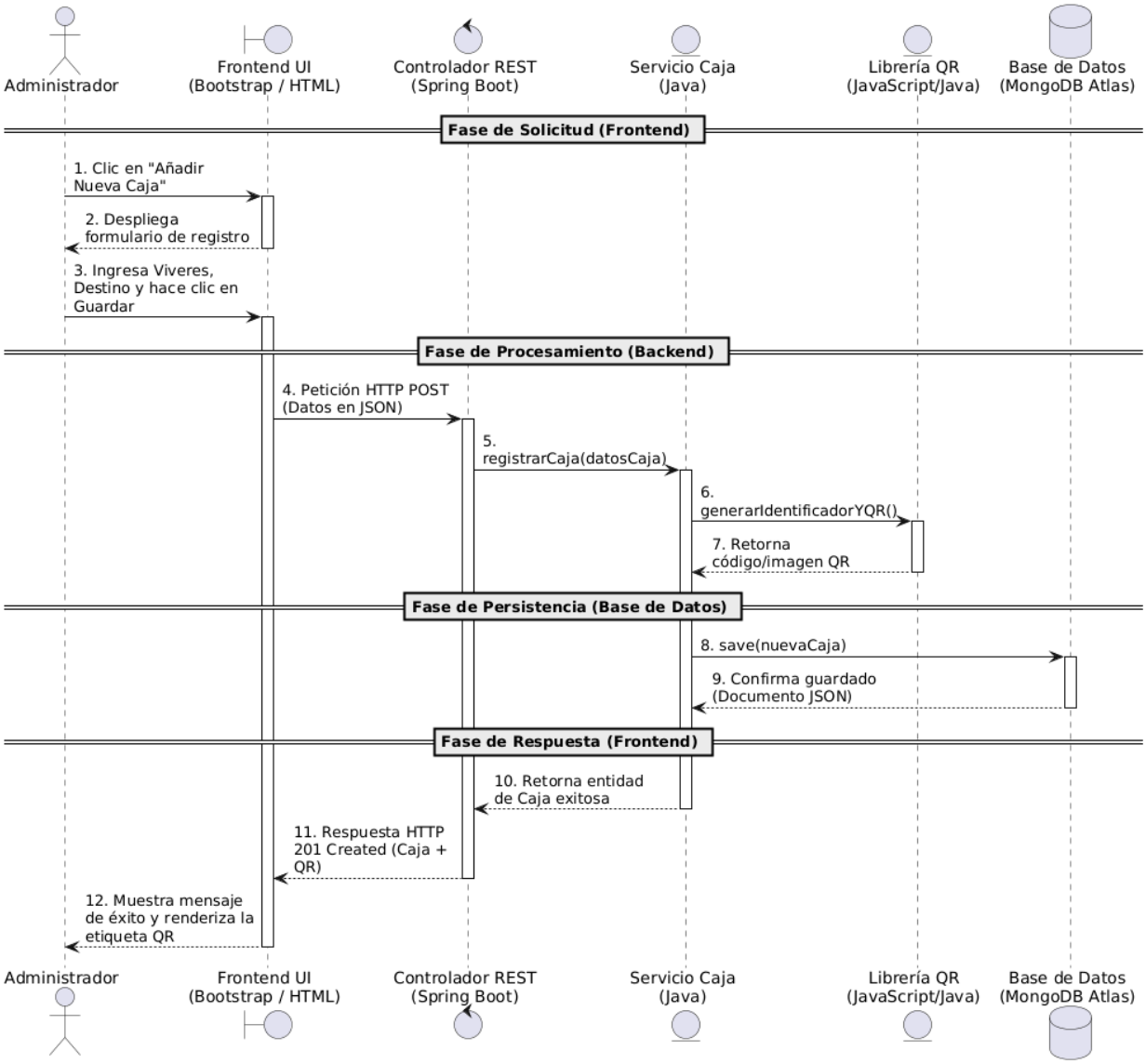
La elección del **Propuesto 1** es totalmente acertada para un sistema de producción. Aunque requiere más clases e interfaces al inicio, garantiza una arquitectura limpia, un control de acceso seguro por rol (impidiendo que usuarios tipo Voluntario invoquen métodos de administración) y una mayor facilidad de mantenimiento y pruebas unitarias a mediano y largo plazo.

2.7.3. Elaborar el diagrama de actividades que modele el comportamiento dinámico de un proceso o tarea propuesto en el sistema software definido

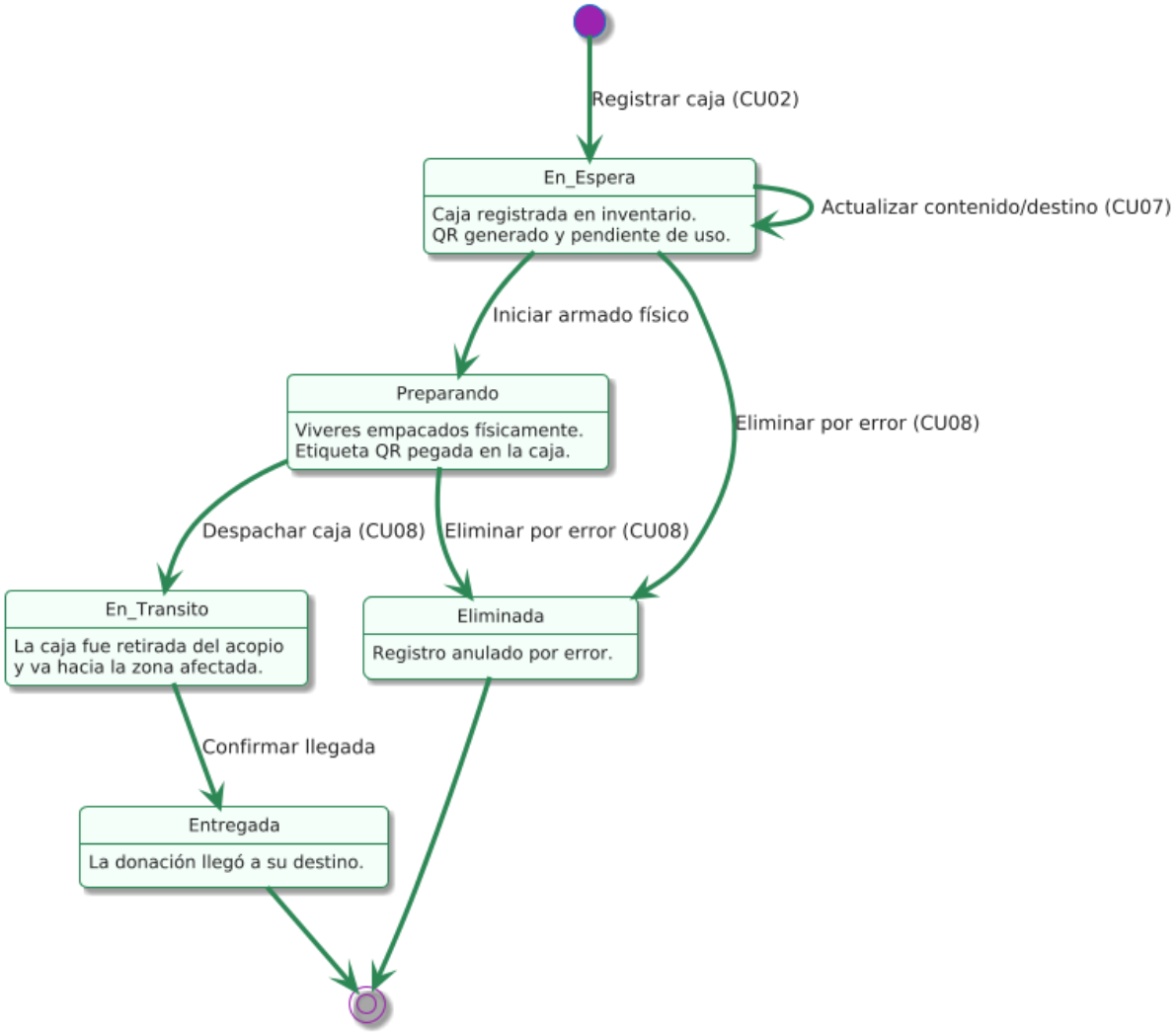

SantanDar – Diagrama de actividades: Registrar caja virtual con QR
Administrador
Sistema (interfaz web + backend)
Base de datos (MongoDB Atlas)
Selecciona "Registrar caja"
Muestra el formulario de registro
Ingresa el contenido y la zona de destino, y confirma
Valida los datos
¿Datos válidos?
no
sí
Muestra los errores y permite corregir
Genera el identificador QR único de la caja
Guarda la caja en el inventario
Muestra el QR y confirma el registro
Visualiza el resultado

1. El administrador selecciona “Registrar caja”.
2. El sistema muestra el formulario.
3. El administrador ingresa el contenido y la zona de destino, y confirma.
4. El sistema valida los datos. Si son inválidos, muestra los errores y el administrador corrige.
5. El sistema genera el identificador QR único de la caja.
6. La base de datos guarda la caja en el inventario.
7. El sistema muestra el QR y confirma el registro.
8. El administrador visualiza el resultado.

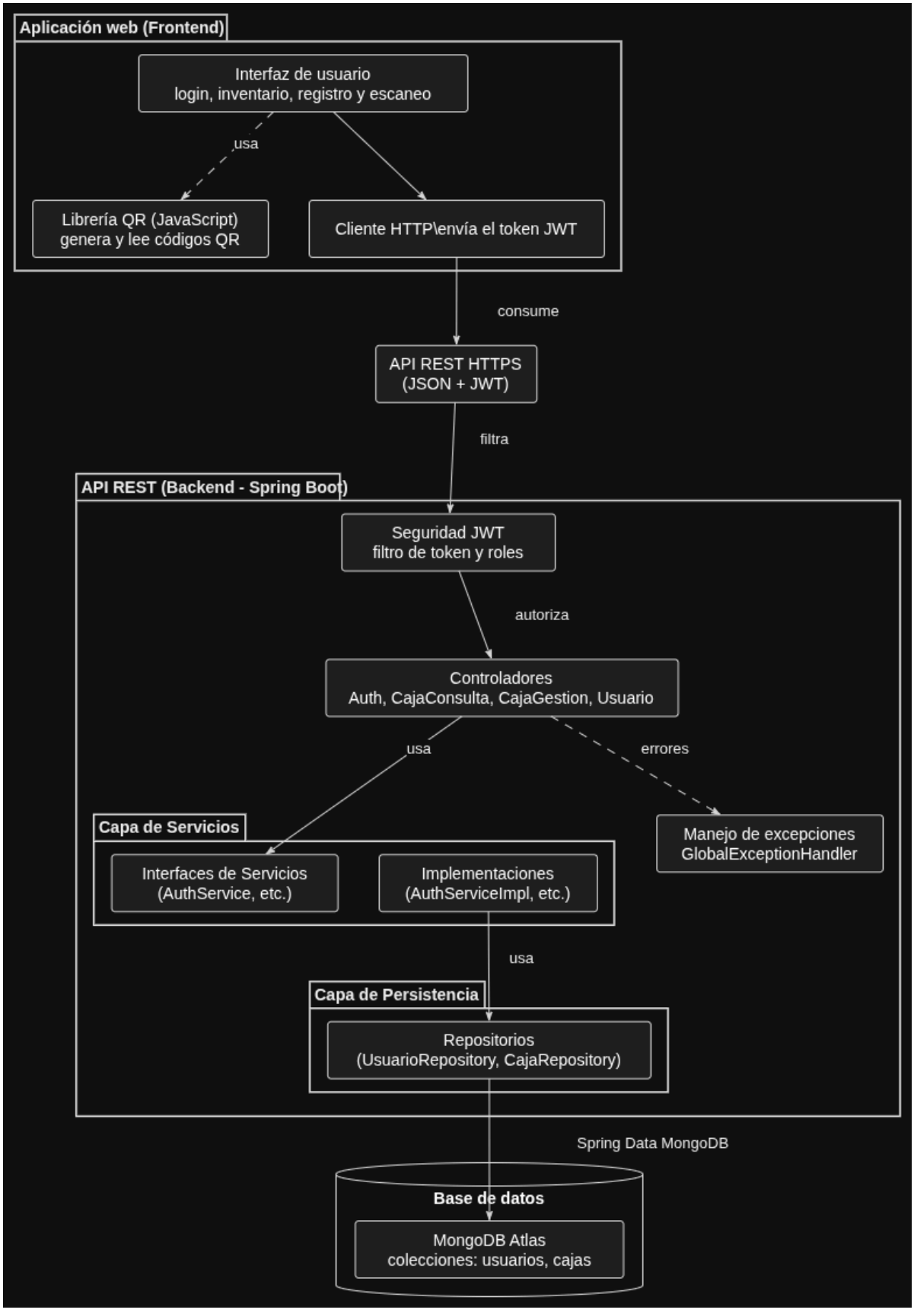
2.7.4. Elaborar los diagramas de secuencia requeridos que representen la interacción entre objetos del sistema a lo largo del tiempo


Administrador
Frontend UI (Bootstrap / HTML)
Controlador REST (Spring Boot)
Servicio Caja (Java)
Librería QR (JavaScript/Java)
Base de Datos (MongoDB Atlas)
Fase de Solicitud (Frontend)
1. Clic en "Añadir Nueva Caja"
2. Despliega formulario de registro
3. Ingresa Viveres, Destino y hace clic en Guardar
Fase de Procesamiento (Backend)
4. Petición HTTP POST (Datos en JSON)
5. registrarCaja(datosCaja)
6. generarIdentificadorYQR()
7. Retorna código/imagen QR
Fase de Persistencia (Base de Datos)
8. save(nuevaCaja)
9. Confirma guardado (Documento JSON)
Fase de Respuesta (Frontend)
10. Retorna entidad de Caja exitosa
11. Respuesta HTTP 201 Created (Caja + QR)
12. Muestra mensaje de éxito y renderiza la etiqueta QR
Administrador
Frontend UI (Bootstrap / HTML)
Controlador REST (Spring Boot)
Servicio Caja (Java)
Librería QR (JavaScript/Java)
Base de Datos (MongoDB Atlas)

2.7.5. Elaborar los diagramas de estados que permitan visualizar los estados posibles asociados a los objetos



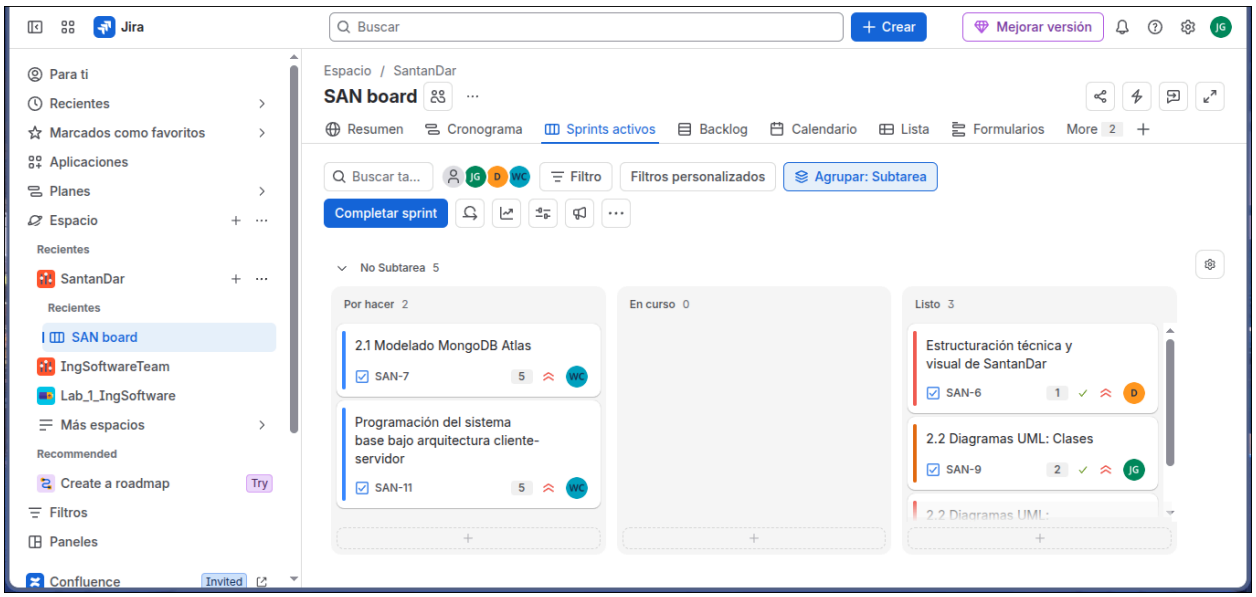
2.7.6. Elaborar el diagrama de componentes donde se modelan los componentes del sistema y las interrelaciones


Aplicación web (Frontend)
Interfaz de usuario
login, inventario, registro y escaneo
usa
Librería QR (JavaScript)
genera y lee códigos QR
Cliente HTTP\envía el token JWT
consume
API REST HTTPS
(JSON + JWT)
filtra
API REST (Backend - Spring Boot)
Seguridad JWT
filtro de token y roles
autoriza
Controladores
Auth, CajaConsulta, CajaGestion, Usuario
usa
errores
Capa de Servicios
Interfaces de Servicios
(AuthService, etc.)
Implementaciones
(AuthServiceImpl, etc.)
Manejo de excepciones
GlobalExceptionHandler
usa
Capa de Persistencia
Repositorios
(UsuarioRepository, CajaRepository)
Spring Data MongoDB
Base de datos
MongoDB Atlas
colecciones: usuarios, cajas

2.8. EVIDENCIAS DEL TRABAJO COLABORATIVO

2.8.1. Usando la herramienta software Jira registre a los integrantes del equipo, y agregue al profesor con el correo electrónico duvansan@correo.uis.edu.co, con base en la información de la entrega anterior complemente el product backlog para su producto o sistema software y regístrelo en la herramienta Jira *(Defina las historias de usuario, crea las épicas y defina la prioridad de los Product Backlog Ítem definidos para su proyecto)*

- *Comparta el enlace de su proyecto en Jira*

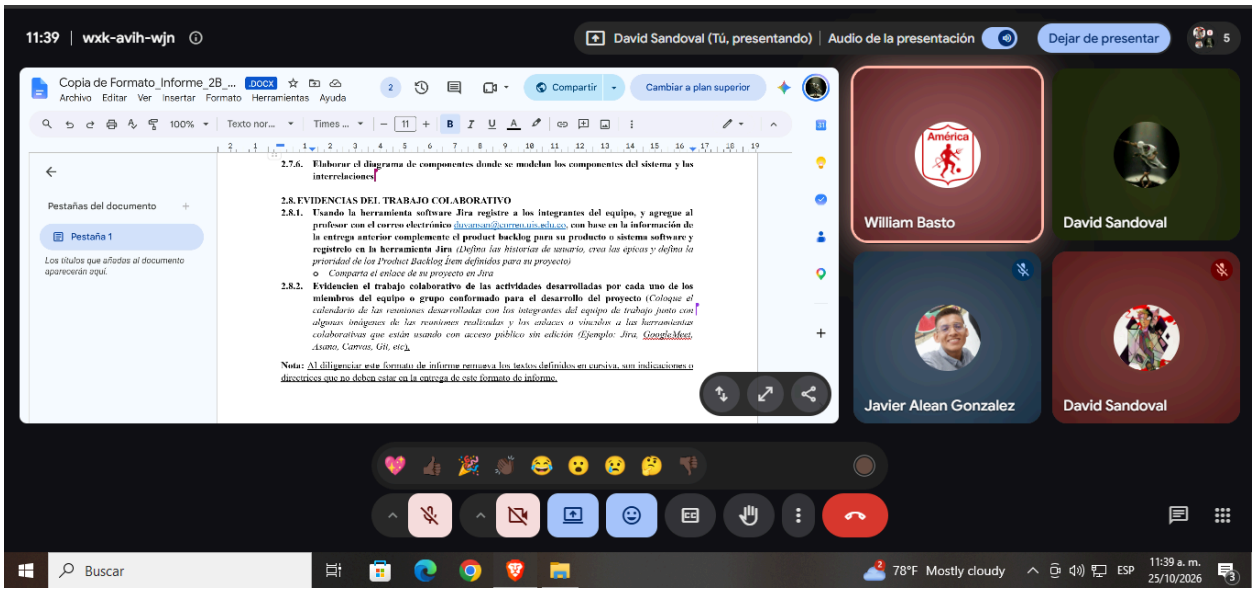


Se realizaron las actualizaciones al Backlog, se marcaron como finalizadas las actividades restantes del Sprint 1 y se da inicio al Sprint 2, además, se marcan las actividades concluidas de este nuevo sprint de duración de 1 semana.

Link del Board del proyecto en Jira: https://javieralean.atlassian.net/jira/software/c/projects/SAN/summary?atlOrigin=eyJpIjoiN2IyN2VhNmU2ZmNhNDUzYmJmMGI1ZGM0YTU1MTZlMGMiLCJwIjoiaiJ9

2.8.2. Evidencien el trabajo colaborativo de las actividades desarrolladas por cada uno de los miembros del equipo o grupo conformado para el desarrollo del proyecto

Evidencia de reuniones asíncronas:



Como herramientas de trabajo colaborativo usamos Google Meet para reuniones asíncronas y Jira como software de gestión de proyectos en el marco de trabajo SCRUM. Además, para los diagramas UML se utilizó LucidChart y PlantText (editor web que usa código PlantUML para generar diagramas UML)

https://meet.google.com/wxk-avih-wjn

